

Recap

We usually lack a global perspective on the objective function f (we don't have a "bird's eye-view").

We can compute the values of f and maybe some of its derivatives at a finite number of points.

Goal: Design an algorithm that can choose a series of points wisely to identify a solution (a local minimum or stationary point) or at least achieve a small value of f without using too much effort

Often, the information about f does not come cheaply, so we prefer algorithms that do not call for this information unnecessarily.

Starting at $w^{(0)}$, the algorithm generates a sequence $\{w^{(k)}\}_{k=0}^{\infty}$ iteratively, using info. about f at $w^{(k)}$ and possible also info. from earlier iterates $w^{(0)}, w^{(1)}, \dots, w^{(k-1)}$

Steepest Descent
Algorithm

Recap

The negative gradient of a function is its steepest descent direction.

$$f(\underbrace{\boldsymbol{w}(k) + \eta \hat{\boldsymbol{v}}}_{\boldsymbol{w}(k+1)}) - f(\boldsymbol{w}(k)) \overset{\substack{\text{1st order Taylor} \\ \text{Approximation}}}{\approx} \underbrace{\eta \nabla f^T(\boldsymbol{w}(k)) \hat{\boldsymbol{v}}}_{\substack{\text{Directional} \\ \text{Derivative at } \boldsymbol{w}(k) \\ \text{along the direction } \hat{\boldsymbol{v}}}} \geq -\eta \|\nabla f(\boldsymbol{w}(k))\|$$

Since $\hat{\boldsymbol{v}}$ is a unit vector, $\hat{\boldsymbol{v}} = -\frac{\nabla f(\boldsymbol{w}(k))}{\|\nabla f(\boldsymbol{w}(k))\|}$

Let the moving distance scale
with the gradient magnitude:

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \eta \frac{\nabla f(\boldsymbol{w}(k))}{\|\nabla f(\boldsymbol{w}(k))\|}$$

$$\eta = \alpha \|\nabla f(\boldsymbol{w}(k))\|$$

$$\boldsymbol{w}(k+1) = \boldsymbol{w}(k) - \alpha \nabla f(\boldsymbol{w}(k)) \quad \text{Steepest Descent Algorithm}$$

Gradient Descent with Exact Line Search

Step 1. Initialize $\mathbf{w}(0)$

$$\mathbf{p}^T(k) \cdot \nabla f(\mathbf{w}(k)) < 0$$

Step 2. For iteration $k = 0, 1, 2, \dots$

Repeat until some stopping criterion is met;

(a) Pick a descent direction $\mathbf{p}(k)$ e.g. $\mathbf{p}(k) = -\nabla f(\mathbf{w}(k))$

(b) Pick the learning rate α by solving $\alpha_k = \arg \min_{\alpha > 0} \phi(\alpha) = f(\mathbf{w}(k) + \alpha \mathbf{p}(k))$

(c) Update $\mathbf{w}(k+1) = \mathbf{w}(k) + \alpha_k \mathbf{p}(k)$

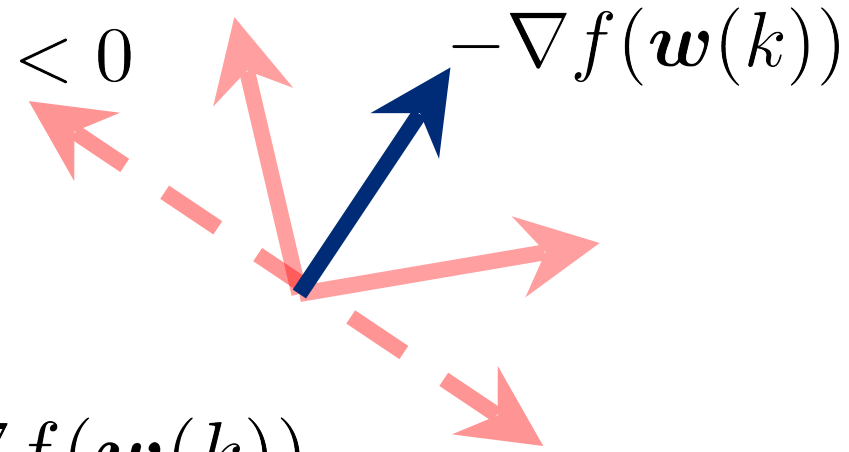
“Exact Line Search”

Termination: Stop when $\|\nabla f(\mathbf{w}(k))\| \leq \epsilon$

In the Line Search Method, we:

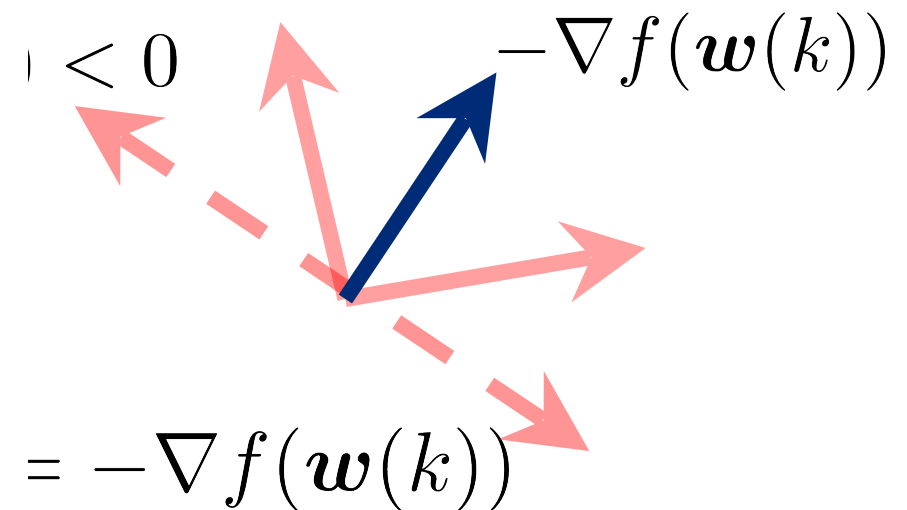
- (1) pick a descent direction $\mathbf{p}(k)$
- (2) decide how far to move along that direction

Ensures *Global Convergence*:
Starting from any initial point, gradient descent with exact line search will lead you toward a stationary point (provided that the gradient is Lipschitz continuous, f is bounded below, etc.)



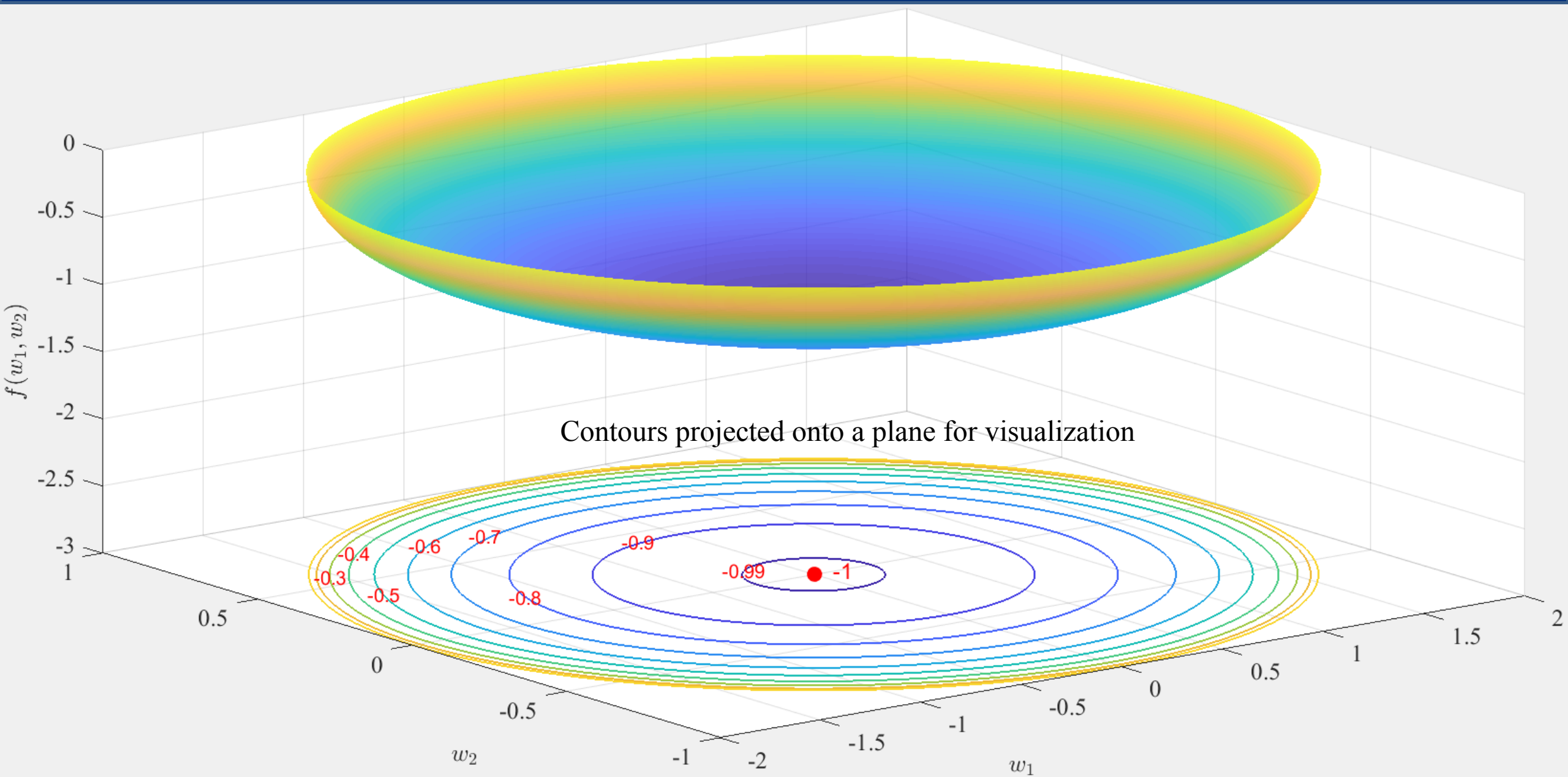
Global Convergence

- At the Strategy Level: *Develop an algorithm that can find a stationary point.*
- At the Operational Level: **Line Search** and *Trust Region Method*
- At the Tactic Level: *Properly design a descent direction and a step size to achieve **Global Convergence** at a low cost*

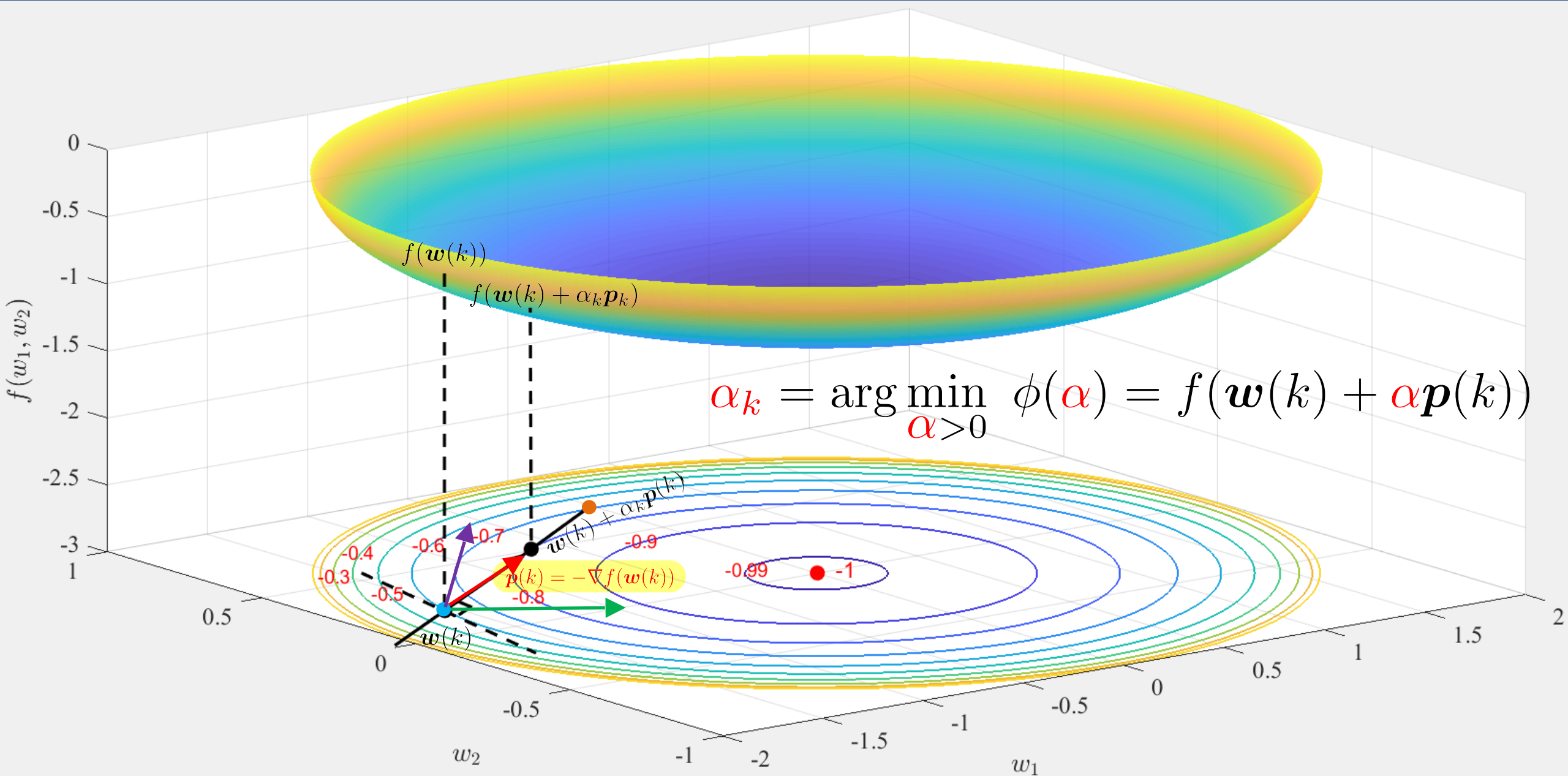


Ensures **Global Convergence**:
Starting from any initial point, gradient descent with exact line search will lead you toward a stationary point (provided that the gradient is Lipschitz continuous, f is bounded below, etc.)

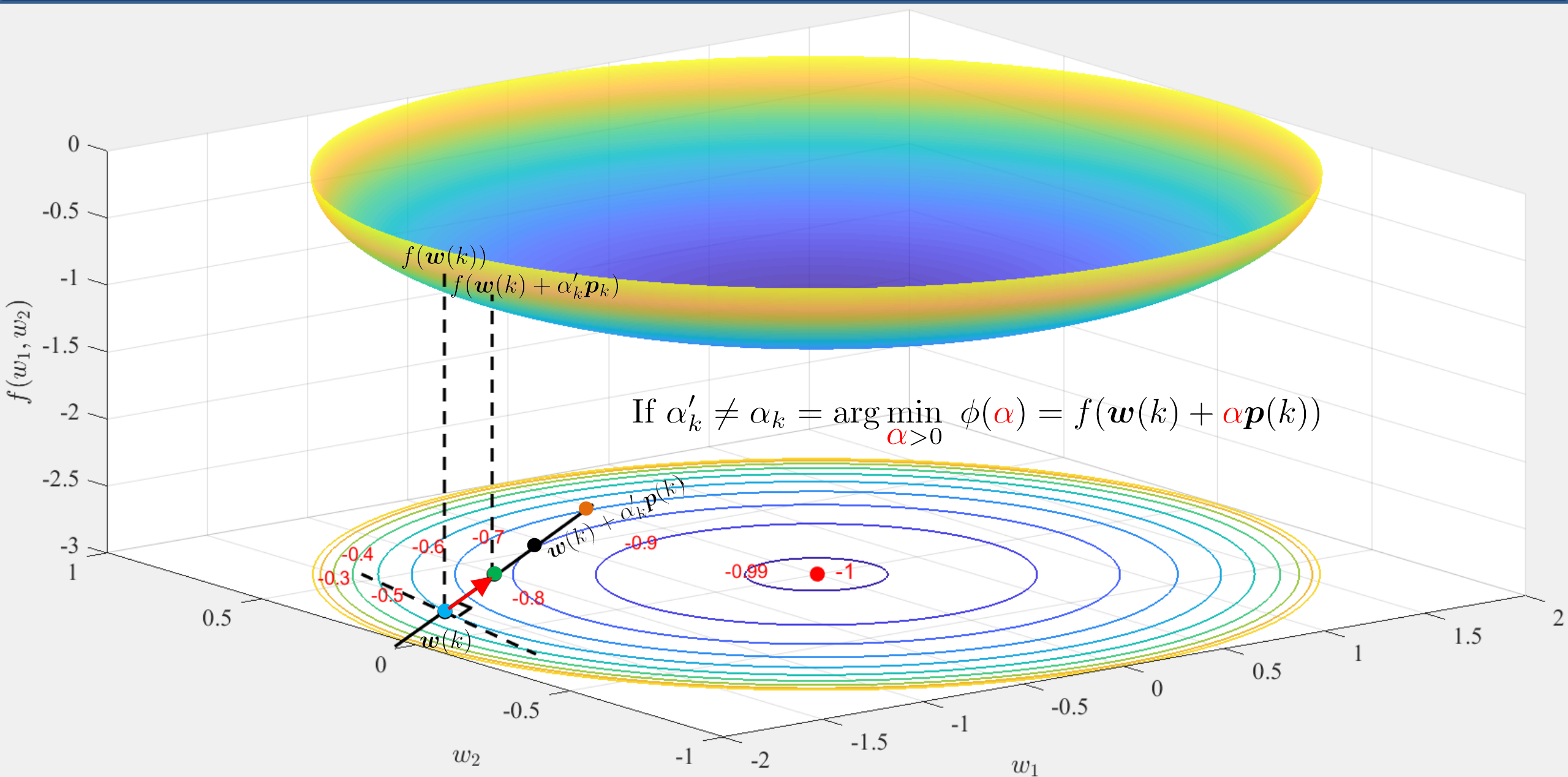
Gradient Descent with Line Search: A Geometric View



Gradient Descent with Exact Line Search



Gradient Descent with Inexact Line Search



How to Choose α ? An Example of Exact Line Search

Example: $\min_{w_1, w_2} f(\mathbf{w}) = w_1 - w_2 + 2w_1w_2 + 2w_1^2 + w_2^2$

In the k^{th} iteration

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha_k \nabla f(\mathbf{w}(k)) \Rightarrow \begin{cases} w_1(k+1) = w_1(k) - \alpha_k(2w_2(k) + 4w_1(k) + 1), \\ w_2(k+1) = w_2(k) - \alpha_k(2w_2(k) + 2w_1(k) - 1). \end{cases}$$

$$\min_{\alpha > 0} \phi(\alpha) = f(\mathbf{w}(k) - \alpha \nabla f(\mathbf{w}(k))) = A\alpha^2 - B\alpha + C \quad \text{where}$$

$$\begin{cases} A = [4w_1(k) + 2w_2(k) + 1]^2 + [6w_1(k) + 4w_2(k)]^2 \geq 0, \\ B = 20w_1^2(k) + 24w_1(k)w_2(k) + 4w_1(k) + 8w_2^2(k) + 2, \\ C = 2w_1^2(k) + 2w_1(k)w_2(k) + w_1(k) + w_2^2(k) - w_2(k). \end{cases}$$

$$\Rightarrow \alpha_k^* = \frac{10w_1^2(k) + 12w_1(k)w_2(k) + 2w_1(k) + 4w_2^2(k) + 1}{[4w_1(k) + 2w_2(k) + 1]^2 + [6w_1(k) + 4w_2(k)]^2}$$

How to Choose α ? An Example of Exact Line Search

Example: $\min_{w_1, w_2} f(\mathbf{w}) = w_1 - w_2 + 2w_1w_2 + 2w_1^2 + w_2^2$

Step 1. Initialize $\mathbf{w}(0)$

Step 2. For iteration $k = 0, 1, 2, \dots$

Compute $\nabla f(\mathbf{w}(k)) = (2w_2(k) + 4w_1(k) + 1, 2w_2(k) + 2w_1(k) - 1)$

$$\alpha_k = \frac{10w_1^2(k) + 12w_1(k)w_2(k) + 2w_1(k) + 4w_2^2(k) + 1}{[4w_1(k) + 2w_2(k) + 1]^2 + [6w_1(k) + 4w_2(k)]^2}$$

Check if $\|\nabla f(\mathbf{w}(k))\| \leq \epsilon$, then stop.

Otherwise, update:

$$\mathbf{w}(k+1) = \mathbf{w}(k) - \alpha_k \nabla f(\mathbf{w}(k))$$

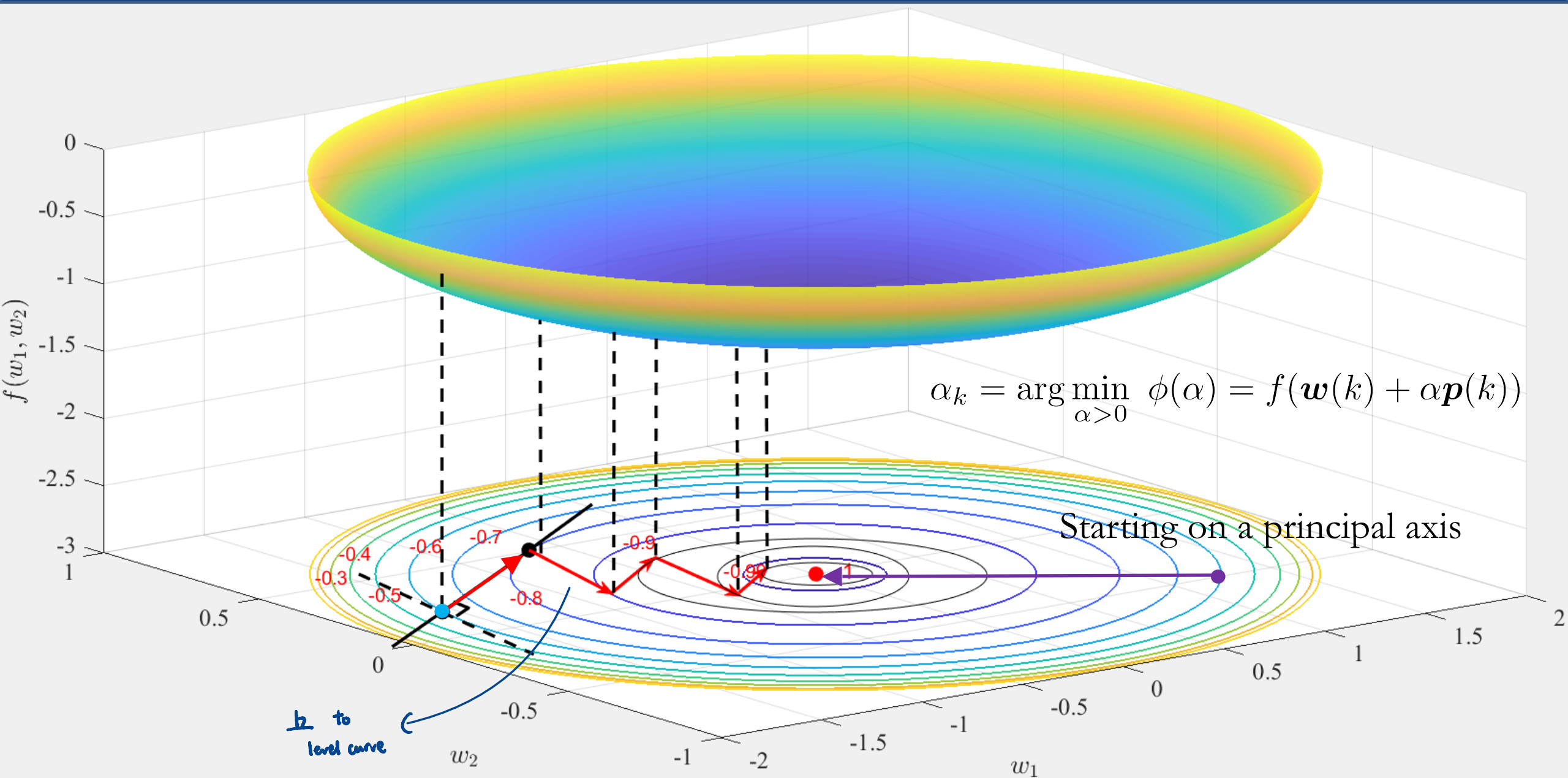
E.g., $\mathbf{w}(0) = \text{rand}(1, 2)$ and $\epsilon = 10^{-7}$

It takes about 20 iterations to converge to $\mathbf{w}^* = (-1, 1.5)$

How to Choose α ? An Example of Exact Line Search

k	$\mathbf{w}(k)$		$\nabla f(\mathbf{w}(k))$		$\ \nabla f(\mathbf{w}(k))\ $	α_k	$f(\mathbf{w}(k))$
0	0.957507	0.964889	6.759804	2.844791	7.334016	0.194964	4.605042
1	-0.36041	0.410258	0.378882	-0.9003	0.976777	1.148322	-0.63829
2	-0.79549	1.444094	0.706239	0.297213	0.76623	0.194964	-1.18609
3	-0.93318	1.386148	0.039584	-0.09406	0.10205	1.148322	-1.24332
4	-0.97863	1.494159	0.073785	0.031052	0.080053	0.194964	-1.2493
5	-0.99302	1.488105	0.004136	-0.00983	0.010662	1.148322	-1.24993
6	-0.99777	1.49939	0.007709	0.003244	0.008364	0.194964	-1.24999
7	-0.99927	1.498757	0.000432	-0.00103	0.001114	1.148322	-1.25
8	-0.99977	1.499936	0.000805	0.000339	0.000874	0.194964	-1.25
9	-0.99992	1.49987	4.51E-05	-0.00011	0.000116	1.148322	-1.25
10	-0.99998	1.499993	8.41E-05	3.54E-05	9.13E-05	0.194964	-1.25
11	-0.99999	1.499999	3.75E-05	5.75E-05	5.91E-05	1.148355	-1.25
12	-1	1.499999	3.75E-05	5.75E-05	5.91E-05	0.194951	-1.25
13	-1	1.499999	3.75E-05	5.75E-05	5.91E-05	1.146678	-1.25
14	-1	1.499999	3.75E-05	5.75E-05	5.91E-05	0.195749	-1.25
15	-1	1.5	4.88E-08	-1.25E-07	1.35E-07	1.0759	-1.25
16	-1	1.5	1.09E-07	3.95E-08	1.16E-07	0.157568	-1.25
17	-1	1.5	2.78E-08	-7.21E-09	2.87E-08	---	-1.25

Exact Line Search: Why the Path Zigzags



Inexact Line Search

Example: $\min_{w_1, w_2} f(\mathbf{w}) = w_1 - w_2 + 2w_1w_2 + 2w_1^2 + w_2^2$

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f(\mathbf{w})}{\partial w_1}, \frac{\partial f(\mathbf{w})}{\partial w_2} \right) = (1 + 2w_2 + 4w_1, -1 + 2w_1 + 2w_2) = 0 \Rightarrow w_1 = -1, w_2 = 3/2$$

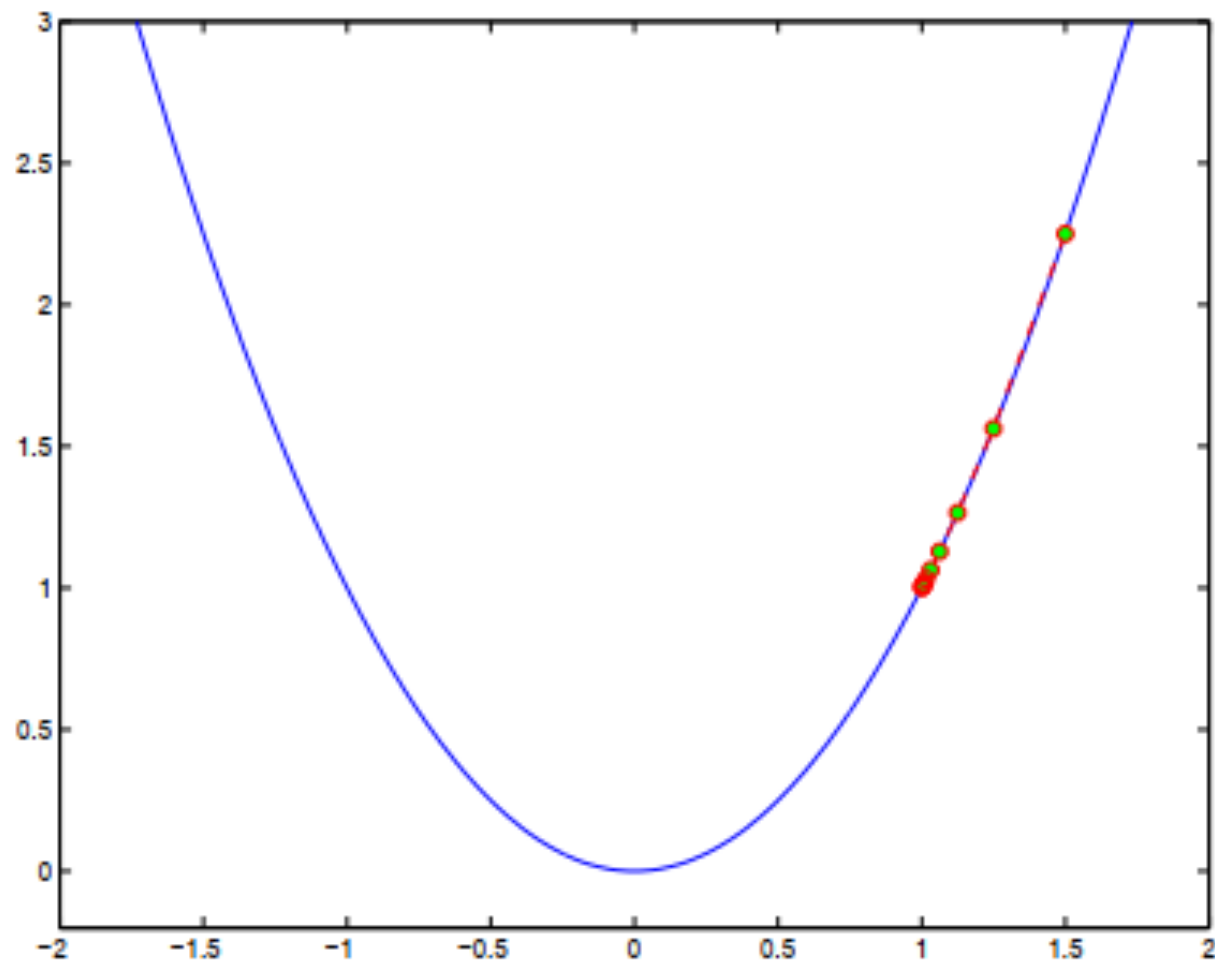
In practice, exact line search is often unavailable in closed form, or too costly to perform at every iteration.

$$\alpha_k = \arg \min_{\alpha > 0} \phi(\alpha) = f(\mathbf{w}(k) + \alpha \mathbf{p}(k))$$

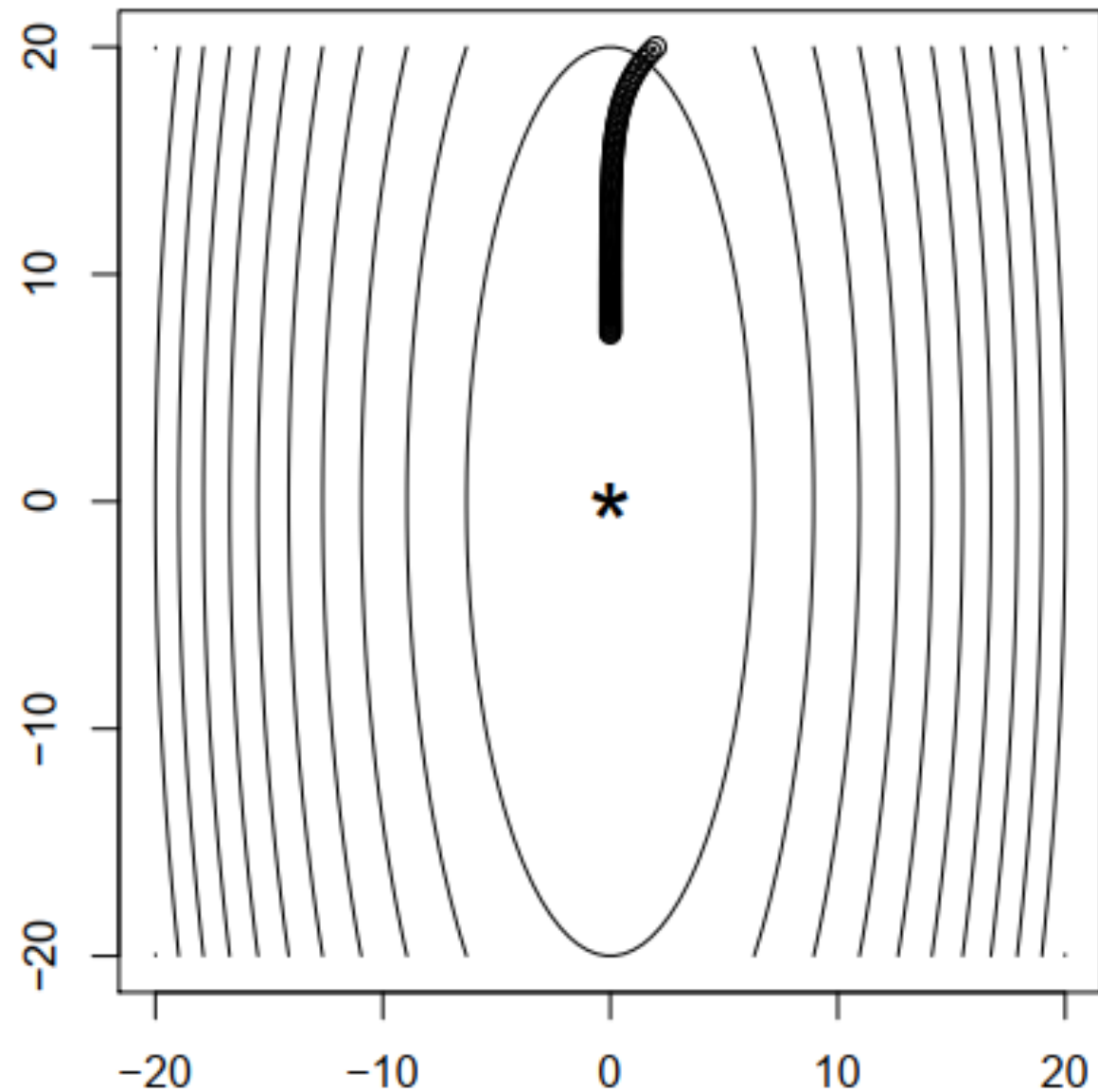
We need to pick a suitable α_k in each iteration with the goal of: ensuring global convergence

The challenges in finding a good α_k lie in avoiding a step size that is either too small or too large

Examples of Bad Choice of α_k (Too Small)

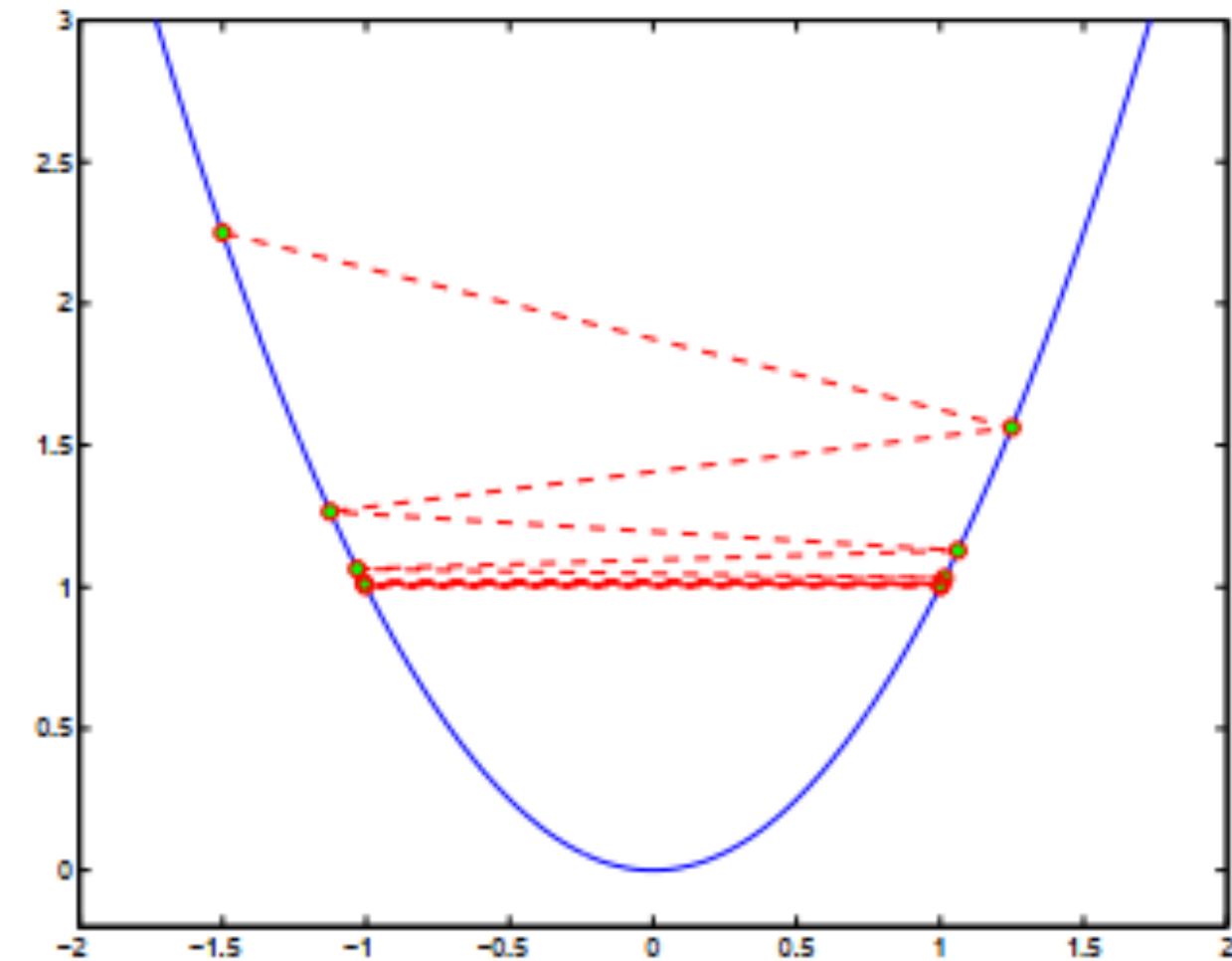


$$f(w) = w^2, \quad w(k+1) = w(k) + \alpha_k p(k)$$
$$w(0) = 1.5, \quad p(k) = -1, \quad \alpha_k = \frac{1}{2^{k+1}}$$



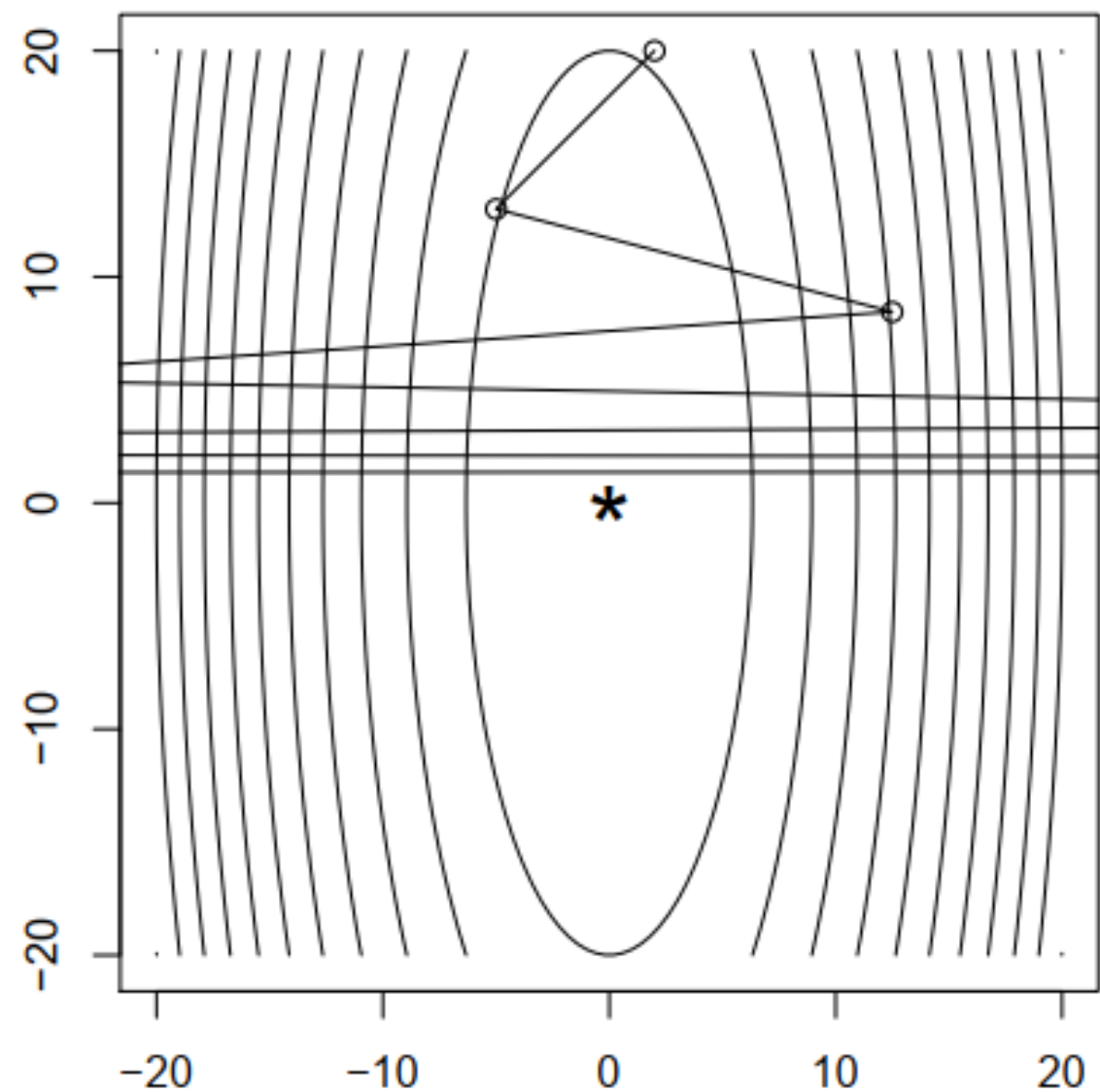
$$f(\mathbf{w}) = (10w_1^2 + w_2^2)/2$$

Examples of Bad Choice of α_k (Too Large)



$$f(w) = w^2, \quad w(k+1) = w(k) + \alpha_k p(k)$$

$$w(0) = 2, \quad p(k) = (-1)^{k+1}, \quad \alpha_k = 2 + \frac{3}{2^{k+1}}$$



$$f(\mathbf{w}) = (10w_1^2 + w_2^2)/2$$

Inexact Line Search Methods

In practice, you will seldom solve the line search problem in exact form... Try to ensure *sufficient* decrease (of the objective function value) in the line search step without spending time to solve it to optimality.

General Procedure of Inexact Line Search

$$\arg \min_{\alpha > 0} \{ \phi(\alpha) = f(\mathbf{w}(k) + \alpha \mathbf{p}(k)) \}$$

- Formulate a criterion that assures that the step size is neither too small nor too large
- Pick a good initial step size $\alpha^{(0)}$
- Construct a sequence of updates of the step size $\{\alpha^{(t)}\}_{t=0,1,\dots}$ that satisfies the above criterion after very few iterations

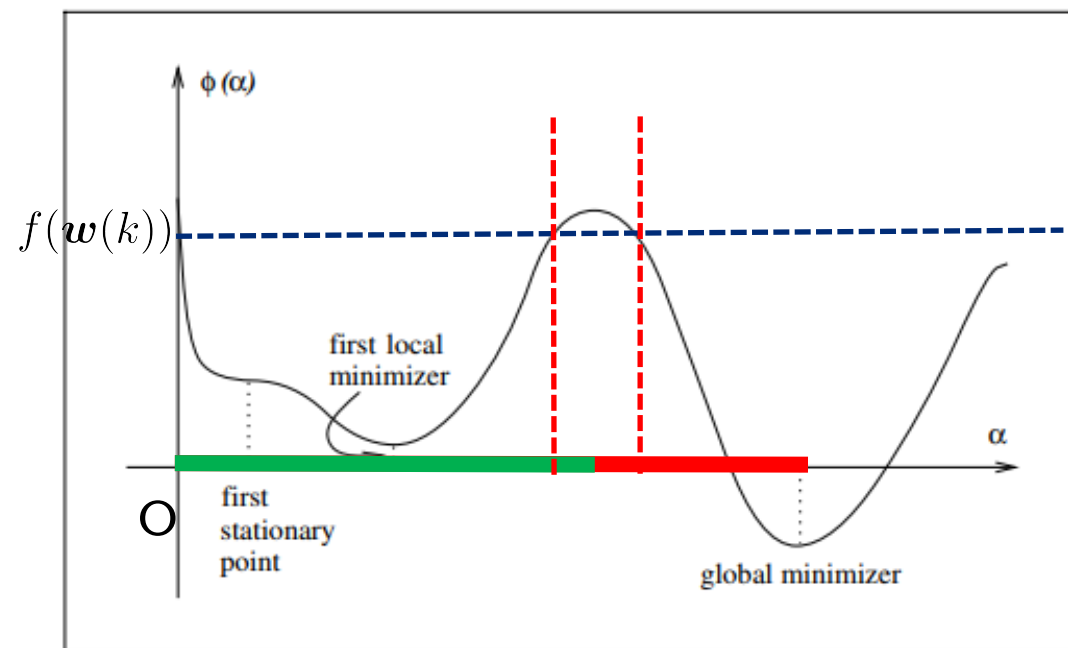


Figure 3.1 The ideal step length is the global minimizer.

Inexact Line Search Methods: Backtracking

Backtracking Line Search: Given the current point $w(k)$ and descent direction $p(k)$

$$\arg \min_{\alpha > 0} \{ \phi(\alpha) = f(w(k) + \alpha p(k)) \}$$

Are there any problems with this method?

1. Initialize $\alpha^{(0)} = 1$ (or some positive number) and set $t = 0$

2. Until $f(w(k) + \alpha^{(t)} p(k)) < f(w(k))$ do the following:

(i) Set $\alpha^{(t+1)} = \rho \alpha^{(t)}$, where $\rho \in (0, 1)$, e.g., $\rho = 1/2$

(ii) Increase t by 1

3. Set $\alpha_k = \alpha^{(t)}$

It does not prevent the step size from being too large

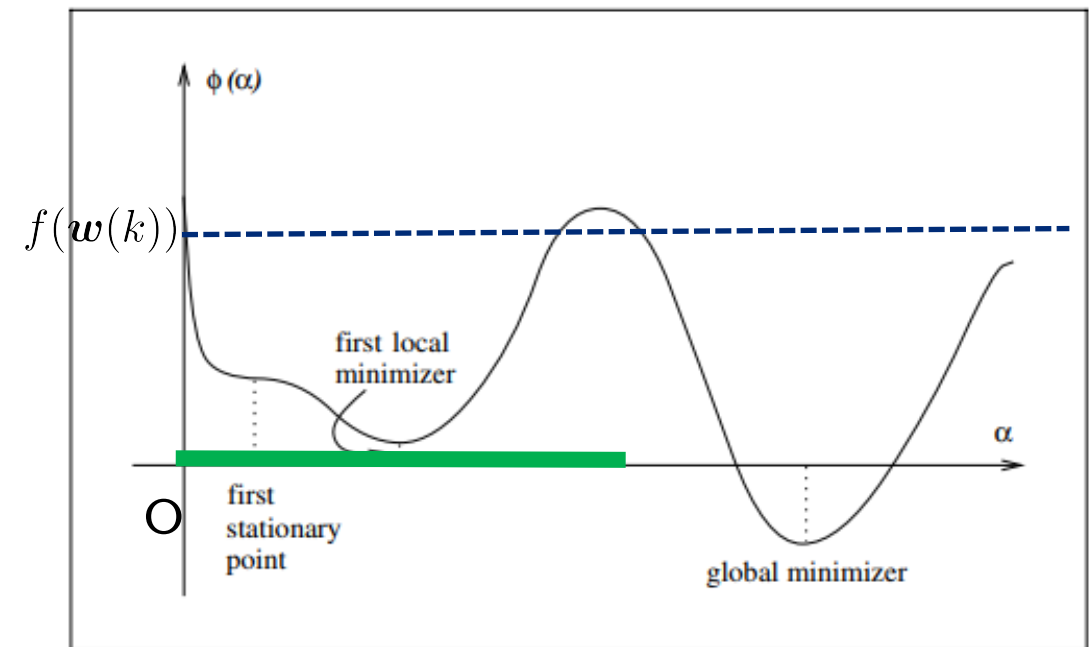


Figure 3.1 The ideal step length is the global minimizer.

Inexact Line Search Methods: Backtracking

Backtracking Line Search: Given the current point $w(k)$ and descent direction $p(k)$

$$\arg \min_{\alpha > 0} \{ \phi(\alpha) = f(w(k) + \alpha p(k)) \}$$

1. Initialize $\alpha^{(0)} = 1$ (or some positive number) and set $t = 0$
2. Until $f(w(k) + \alpha^{(t)} p(k)) < f(w(k))$ do the following:

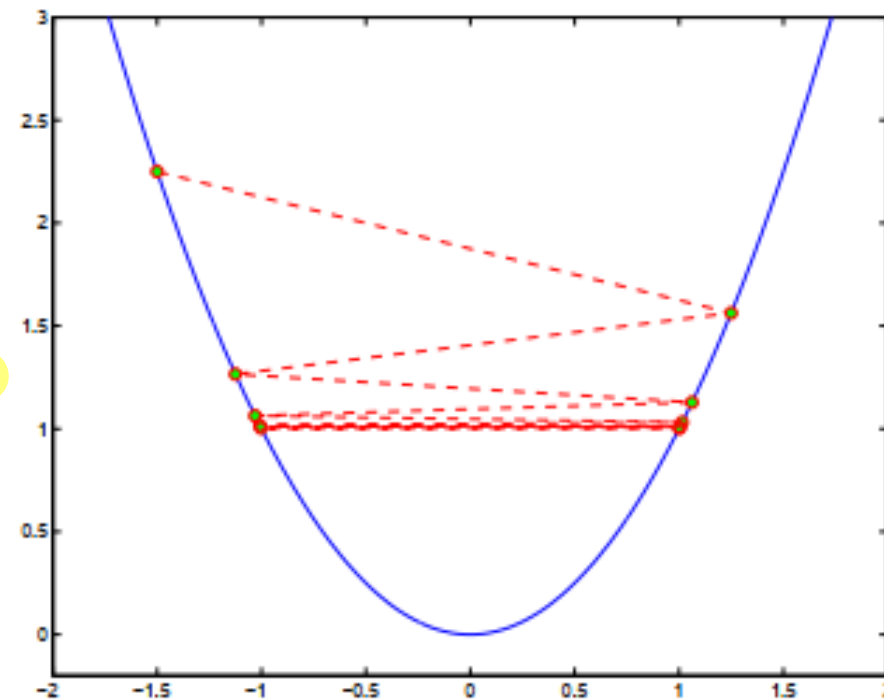
(i) Set $\alpha^{(t+1)} = \rho \alpha^{(t)}$, where $\rho \in (0, 1)$, e.g., $\rho = 1/2$

(ii) Increase t by 1

3. Set $\alpha_k = \alpha^{(t)}$

The reduction in the objective function value is not enough

Are there any problems with this method?



Inexact Line Search Methods: Backtracking with Armijo Condition

* does not prevent unnecessarily small step

$f(\mathbf{w}(k) + \alpha^{(t)} \mathbf{p}(k)) < f(\mathbf{w}(k))$ becomes

$$f(\mathbf{w}(k) + \alpha^{(t)} \mathbf{p}(k)) < f(\mathbf{w}(k)) + \alpha^{(t)} c_1 \cdot \mathbf{p}^T(k) \nabla f(\mathbf{w}(k)) \triangleq l(\alpha) < f(\mathbf{w}(k))$$

$c_1 \in (0, 1)$, e.g., $c_1 = 0.001$ Because $\mathbf{p}(k)$ is a descent direction, so $\mathbf{p}^T(k) \nabla f(\mathbf{w}(k)) < 0$

We require that the achieved reduction of f be at least a fixed fraction c of the reduction promised by the first-order Taylor approximation of f at $\mathbf{w}_k$

1. Initialize $\alpha^{(0)} > 0$ (e.g., 0.5) and set $t = 0$
2. Until $f(\mathbf{w}(k) + \alpha^{(t)} \mathbf{p}(k)) < l(\alpha)$ do the following:
 - (i) Set $\alpha^{(t+1)} = \rho \alpha^{(t)}$, where $\rho \in (0, 1)$, e.g., $\rho = 1/2$
 - (ii) Increase t by 1
3. Set $\alpha_k = \alpha^{(t)}$

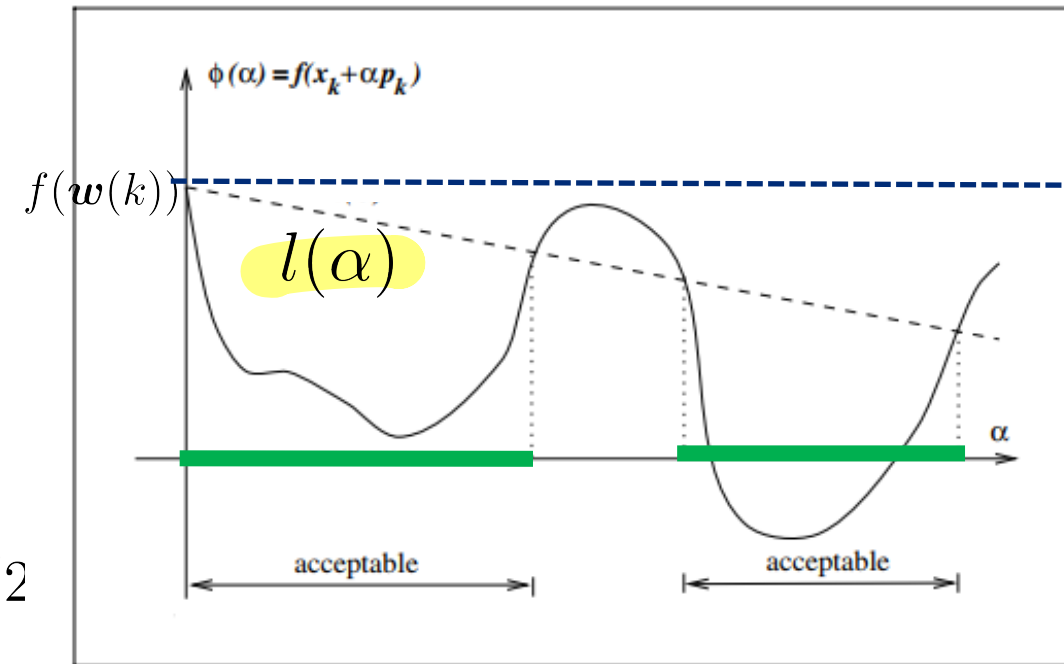


Figure 3.3 Sufficient decrease condition.

Inexact Line Search Methods: Wolfe Condition

The Backtracking Line Search with the Armijo condition is simple and popular for many algorithms, but for some algorithms (e.g., Quasi-Newton), we need a stronger condition

$$\left. \begin{aligned} \text{Armijo: } f(\mathbf{w}(k) + \alpha^{(t)} \mathbf{p}(k)) &\leq f(\mathbf{w}(k)) + \alpha^{(t)} c_1 \cdot \mathbf{p}^T(k) \nabla f(\mathbf{w}(k)) \\ \text{Curvature: } \nabla f(\mathbf{w}(k) + \alpha^{(t)} \mathbf{p}(k))^T \mathbf{p}(k) &\geq c_2 \cdot \mathbf{p}^T(k) \nabla f(\mathbf{w}(k)) \end{aligned} \right\} \text{Wolfe Conditions}$$

where $0 < c_1 < c_2 < 1$ e.g., $c_1 = 10^{-4}$, $c_2 = 0.9$

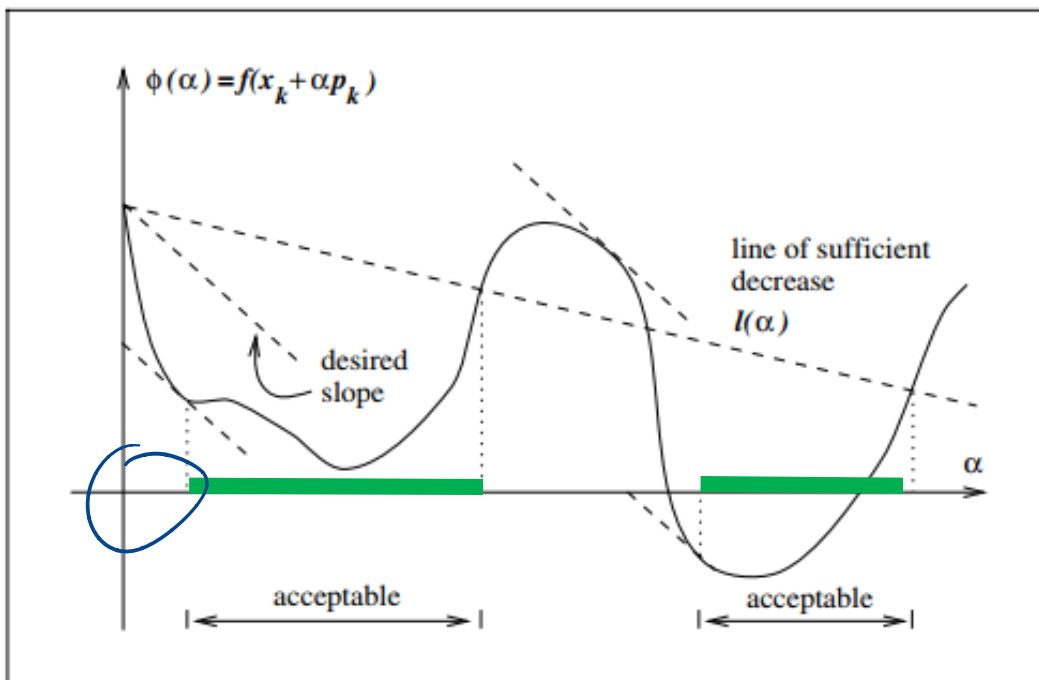


Figure 3.5 Step lengths satisfying the Wolfe conditions.

Under standard regularity assumptions, suitable descent directions combined with Wolfe steps ensure global convergence to stationarity.

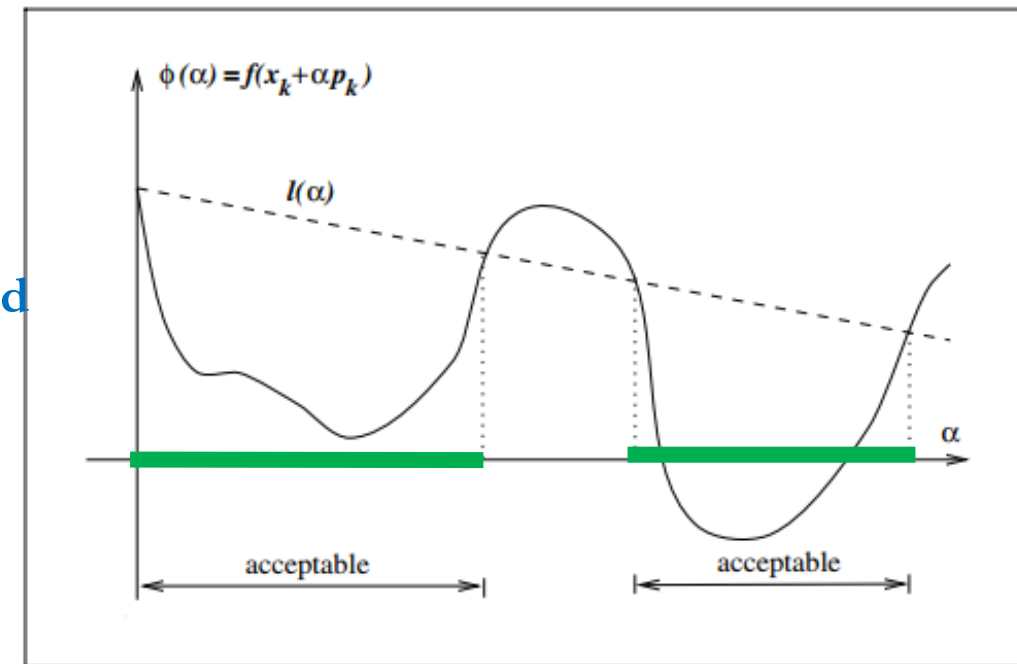


Figure 3.3 Sufficient decrease condition.

Wolfe Conditions in the $\phi(\alpha)$ View: One test for height; one test for slope

$$\phi(\alpha) = f(\mathbf{w}(k) + \alpha \mathbf{p}(k)) \quad \phi(0) = f(\mathbf{w}(k))$$

$$\phi'(\alpha) = \nabla f(\mathbf{w}(k) + \alpha \mathbf{p}(k))^T \mathbf{p}(k) \quad \phi'(0) = \nabla f(\mathbf{w}(k))^T \mathbf{p}(k) < 0$$

Because $\mathbf{p}(k)$ is a descent direction, so the initial directional slope is negative

Armijo: The “Height” Test

$$\phi(\alpha) \leq \phi(0) + c_1 \alpha^{(t)} \phi'(0)$$

$$c_1 \underbrace{[-\alpha^{(t)} \phi'(0)]}_{\text{1st order predicted decrease}} \leq \underbrace{\phi(0) - \phi(\alpha)}_{\text{Actual decrease}}$$

1st order
predicted
decrease

Actual decrease

Have we gone down by enough?

Reject an aggressive step that delivers too little decrease.

Armijo checks the receipt: did we receive enough decrease for the movement we made?

Curvature checks the speedometer: are we still rushing downhill almost as fast as when we started?

Curvature: The “Slope” Test

$$\phi'(\alpha) \geq c_2 \phi'(0)$$

e.g., $\overset{\text{initial}}{\phi'(0)} = -10$, $c_2 = 0.9$, $\Rightarrow \phi'(\alpha) \geq -9$

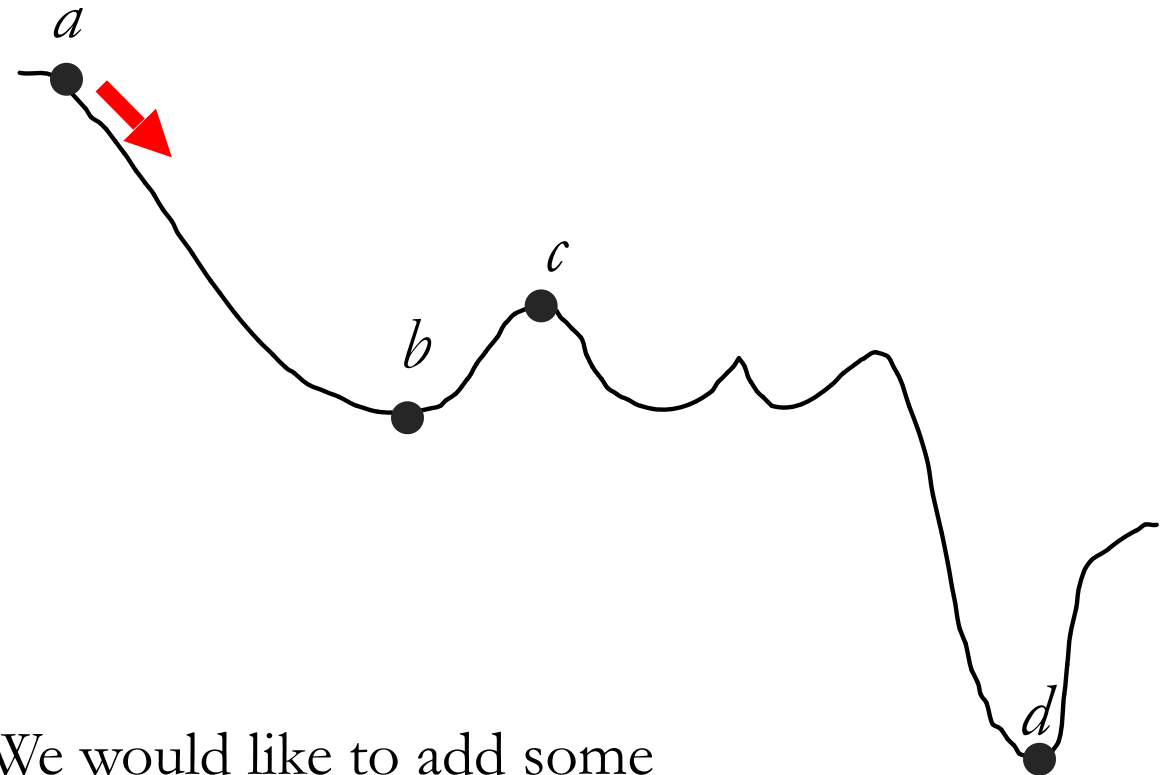
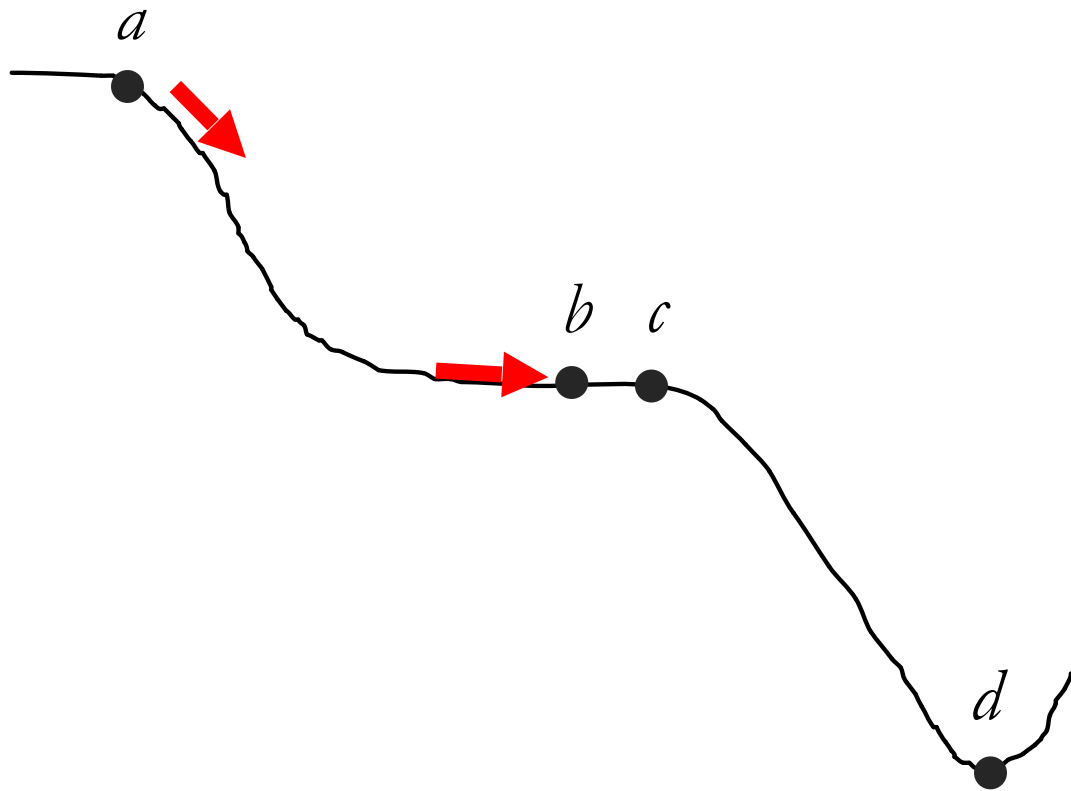
$$\begin{cases} \phi'(\alpha_1) = -9.99 & \Rightarrow \text{too early, reject,} \\ \phi'(\alpha_2) = -5 & \Rightarrow \text{far enough, accept,} \\ \phi'(\alpha^*) = 0 & \Rightarrow \text{Exact line search.} \end{cases}$$

Has the downhill slope flattened enough?

Some Issues of Steepest Descent

So far, the descent direction is given; in particular, it is the negative gradient, the steepest descent direction

Although it is a very intuitive descent direction, it suffers from several issues:



We would like to add some “momentum” to our gradient descent algorithm.

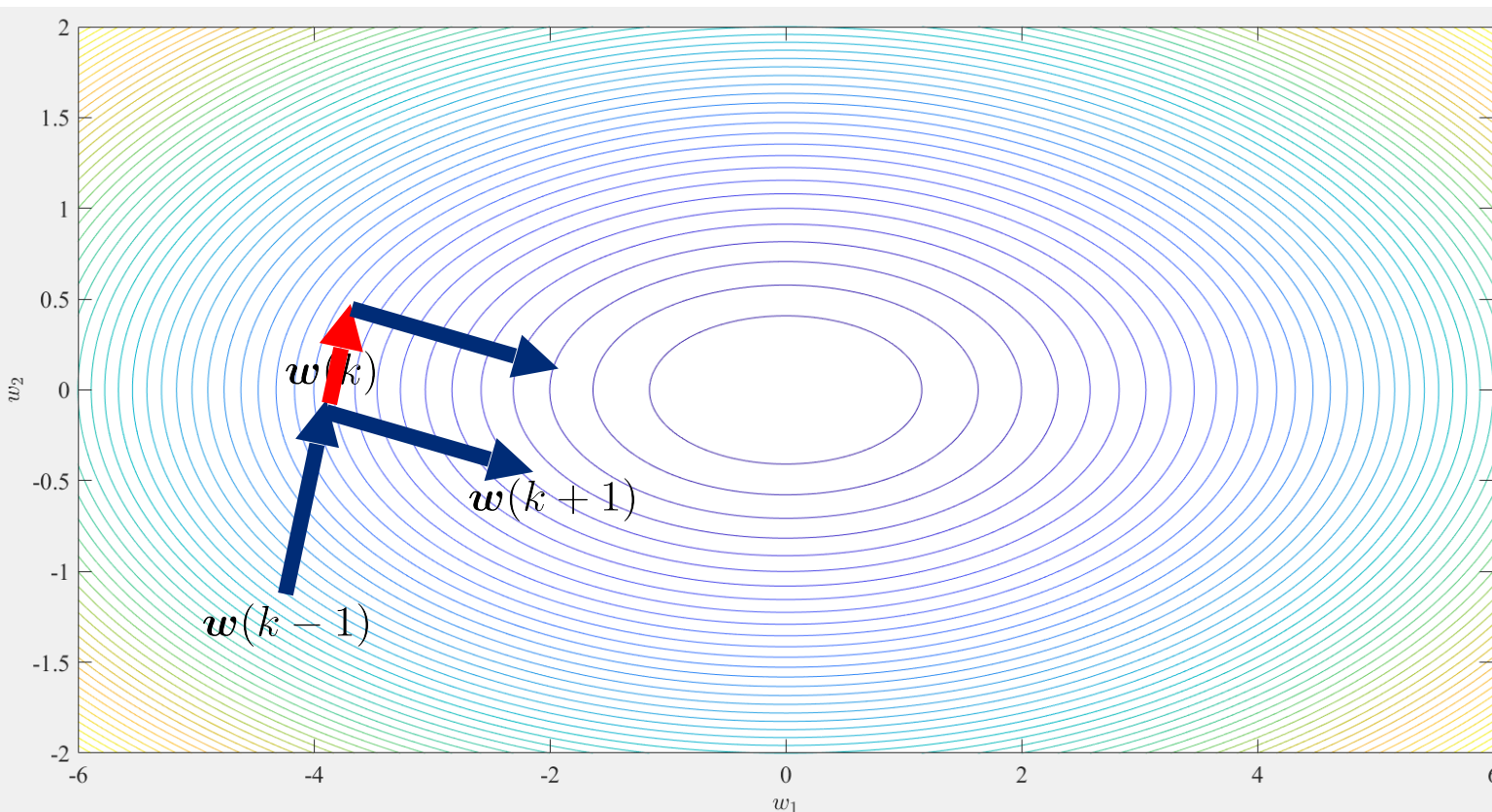
Steepest Descent with Momentum

In the plain vanilla version of the steepest descent algorithm, we have:

$$\mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k) \quad \text{“Momentum”}$$

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mathbf{v}(k+1)$$

$$\mathbf{v}(0) = \mathbf{0}$$



$\tau = 0.8, 0.9$ Oftentimes. It has the interpretation of momentum retention

Gradient Descent with Momentum

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mathbf{v}(k+1) \text{ where } \mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k), \quad \mathbf{v}(0) = \mathbf{0}$$

With momentum, current and previous gradients are combined using exponentially decaying weights

$$\mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k)$$

*furthest gradients
have least impact on gradient*

$$= -\alpha \nabla f(\mathbf{w}(k)) + \tau [-\alpha \nabla f(\mathbf{w}(k-1)) + \tau \mathbf{v}(k-1)]$$

$$= -\alpha \nabla f(\mathbf{w}(k)) - \tau \alpha \nabla f(\mathbf{w}(k-1)) + \tau^2 \mathbf{v}(k-1)$$

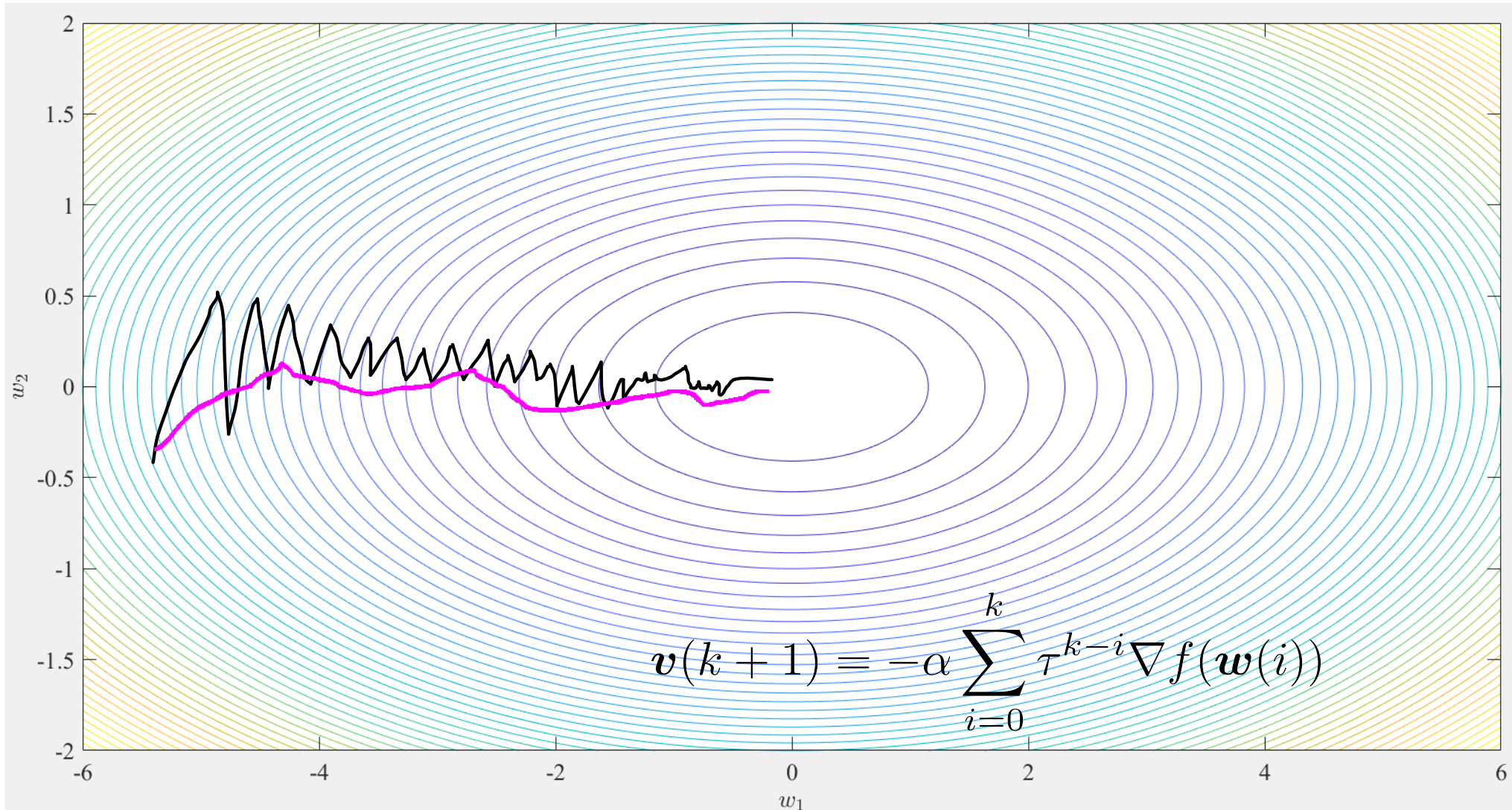
$$= -\alpha \nabla f(\mathbf{w}(k)) - \tau \alpha \nabla f(\mathbf{w}(k-1)) + \tau^2 [-\alpha \nabla f(\mathbf{w}(k-2)) + \tau \mathbf{v}(k-2)]$$

$$= \underbrace{-\alpha \nabla f(\mathbf{w}(k))}_{\text{current gradient}} - \underbrace{\tau \alpha \nabla f(\mathbf{w}(k-1)) - \tau^2 \alpha \nabla f(\mathbf{w}(k-2)) + \tau^3 \mathbf{v}(k-2)}_{\text{previous gradients}}$$

$$= -\alpha \sum_{i=0}^k \tau^{k-i} \nabla f(\mathbf{w}(i)) + \tau^{k+1} \mathbf{v}(0) = -\alpha \sum_{i=0}^k \tau^{k-i} \nabla f(\mathbf{w}(i)), \quad \because \mathbf{v}(0) = \mathbf{0}$$

Gradient Descent with Momentum

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mathbf{v}(k+1) \quad \text{where} \quad \mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k), \quad \mathbf{v}(0) = \mathbf{0}$$



Gradient Descent with Momentum

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \mathbf{v}(k+1) \quad \text{where } \mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k), \quad \mathbf{v}(0) = \mathbf{0}$$

$$\text{where } \mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k) + \tau \mathbf{v}(k)) + \tau \mathbf{v}(k)$$

Nesterov's Accelerated Gradient Descent (NAG): Look Ahead First

Another implementation of NAG

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \tau [\mathbf{v}(k+1) - \mathbf{v}(k)] + \mathbf{v}(k+1)$$

$$\text{where } \mathbf{v}(k+1) = -\alpha \nabla f(\mathbf{w}(k)) + \tau \mathbf{v}(k)$$

Difference between Adding Momentum and Random Initialization

Random initialization is still “pedaling on the brake” but with different starting point. It won’t let you escape from the current local minimum and/or saddle point. Its improvement relies on learning the geometry of the objective function by RANDOM sampling.

Momentum utilizes the “history” (non-random) to determine the next moving direction.

